

Базы данных
Задания для выполнения расчетно-
графических работ

Введение

Целью решения информационных задач является обеспечение пользователей наиболее полной и достоверной информацией по интересующей его проблеме. Специфика информационных задач связана с необходимостью создания, длительного хранения и преобразования больших массивов информации.

Для достижения указанной цели создано большое количество программных средств, ориентированных на какие-либо области применения, и вычислительные средства. Наибольшее практическое применение получили информационные системы, в основе которых лежит использование баз данных (БД). Развитие концепции организации БД привело к возможности создания универсальных систем управления базами данных (СУБД). Основное требование в данной концепции – необходимость разбиения информации на сравнительно небольшое (фиксированное) количество классов однотипных объектов. Количество объектов в каждом классе может быть огромным. Не все предметные области удовлетворяют указанному ограничению: количество классов может быть огромным, а количество объектов в каждом классе небольшим. Наиболее характерными примерами таких систем являются информационно-поисковые системы (ИПС). Необходимо отметить, что, абстрагируясь от каких-либо специфических характеристик объектов, можно объединить их в небольшое количество классов и в последующем использовать для создания информационной системы какую-либо СУБД, но платой за это будет снижение качества информации.

Решению о создании БД и закупки для этой цели ЭВМ и СУБД должен предшествовать анализ прикладной области. Первоначально необходимо выяснить целесообразность создания БД: будет ли экономический эффект от ее использования окупать затраты на создание и сопровождение. К этому анализу необходимо вернуться после построения проекта системы, когда оценки по затратам и экономическому эффекту могут быть уточнены. При положительном ответе на первый вопрос необходимо выяснить целесообразность использования универсальных СУБД. Как правило, СУБД является довольно громоздкой системой, требующей значительных объемов оперативной и внешней памяти. Кроме того требуются дополнительные затраты времени на преобразование и передачу данных. Использование универсальных СУБД не является целесообразным, если первичным является прикладное программное обеспечение по обработке данных, а не сами данные, т.е. жизненный цикл данных меньше либо равен жизненному циклу программ. Такая ситуация наиболее часто встречается в информационно-вычислительных системах, когда какие-либо данные необходимы только для работы одной или нескольких связанных между собой программ. В этом случае достаточно использовать один из методов доступа, соответствующий способу обработки данных в программах.

Если данные имеют самостоятельную ценность: большой жизненный цикл данных, использование их в различных приложениях и т.д., то появляется необходимость использования СУБД. На проблему выбора СУБД оказывают

влияние следующие факторы: 1) наличие или возможность приобретения ЭВМ и операционной системы, на которую ориентирована СУБД; 2) соответствие типа модели данных, который выбран при проектировании, и типа СУБД; 3) наличие функциональных возможностей в СУБД, необходимых для реализации приложений; 4) обеспечение многопользовательского режима доступа к данным; 5) соответствие стандарту языков описания и манипулирования данными и т.д. Наиболее важным этапом является построение модели данных прикладной области (схемы базы данных). Правильно построенная схема обеспечивает устойчивость всей системы к различным изменениям в прикладной области в процессе эксплуатации БД, что существенно сокращает затраты на сопровождение. Экономия достигается за счет отсутствия необходимости переписывания старого прикладного программного обеспечения и локальности необходимых преобразований физической организации данных. Отправной точкой для проектирования схемы БД является изучение документооборота в заданной прикладной области.

Типовое задание на проектирование схемы базы данных

В индивидуальном задании предлагается прикладная область, для которой требуется составить проект БД. Кроме того, приведен перечень документов, являющихся источником сведений для построения схемы. В некоторых случаях приводятся комментарии к документам, уточняющие их специфику.

Типовое задание содержит 6 пунктов. Результаты выполнения каждого пункта являются основой для выполнения следующих пунктов и ошибки, допущенные в каком-либо пункте, обязательно проявятся в результирующей схеме БД.

Этапы типового задания:

Этап 1. Для каждого документа выделить содержащиеся в нем элементы данных. Составить их общий перечень. Список может быть расширен за счет рекомендаций пользователя. При этом требуется: а) объединить синонимы в один элемент данных и присвоить ему имя, приемлемое для всех пользователей; б) разъединить омонимы, присвоив каждому элементу данных уникальное имя.

Общее требование к элементам данных следующее:

Элементом данных называется поименованное неизменяемое атомарное данное, не допускающее декомпозиции в прикладной области, с определенным типом и наименованием. Причем наименование должно однозначно определять семантику элемента данных без какого-либо контекста.

Ошибки: 1) *Цех* – этот элемент данных может содержать в себе другие элементы: *начальник цеха*, *выпускаемая продукция* и др., то есть отсутствует семантическая однозначная интерпретация этого элемента (наверное, имелось в виду *название цеха* или *номер цеха*); 2) *Средняя зарплата в отделе* – этот элемент не является первичной информацией, а может быть вычислен, следо-

вательно, в базе данных надо хранить сведения о выплатах каждому сотруднику отдельно, а среднее, минимальное, максимальное значения – вычислять в прикладной программе; 3) *Возраст* – этот элемент имеет изменяемое значение (в системе отсутствует событие, приводящее к необходимости изменения этого значения), нужно использовать элемент "дата рождения", а возраст вычислять в приложении.

Этап 2. Для выделенных в пункте 1 атрибутов построить множество функциональных зависимостей.

Определение. Пусть задана схема отношения R на совокупности атрибутов $U = \{A_1, A_2, \dots, A_n\}$, (возможно R – тип записи для структурированной модели данных). Пусть X и Y – некоторые подмножества из множества атрибутов U . Будем говорить, что X функционально определяет Y ($X \rightarrow Y$), если в любой реализации r схемы R не могут присутствовать два кортежа $t, u \in r$, такие что $t[X] = u[X]$ и $t[Y] \neq u[Y]$.

Полученное множество функциональных зависимостей обозначим F . Из этого множества удаляются транзитивные и частичные зависимости. Для этого должны быть выполнены следующие построения:

Шаг 1. По правилу декомпозиции функциональных зависимостей преобразуем F к виду, где все зависимости имеют один атрибут в правой части.

Шаг 2. По очереди удаляем каждую зависимость из F : $G = F \setminus \{X \rightarrow A_i\}$, и проверяем эквивалентность полученного множества G исходному F (достаточно проверить условие: $A_i \in X_G^+$). Если получено эквивалентное множество, то зависимость не возвращается, если не получено – возвращается.

Шаг 3. Последовательно исключаем по одному атрибуту в левой части каждой функциональной зависимости $X \rightarrow A_i \in F$. Пусть $Z = X \setminus A_i$ и $G = \{F \setminus \{X \rightarrow A_i\}\} \cup \{Z \rightarrow A_i\}$, если полученное множество G эквивалентно F (достаточно проверить условие: $A_i \in Z_F^+$), то атрибут A_i не возвращается, иначе возвращается.

Замечание. Полученное множество зависимостей называется минимальным покрытием. Различный порядок просмотра зависимостей на шагах 2 и 3 может привести к различным результатам. Следовательно, существует несколько минимальных покрытий исходного множества функциональных зависимостей. В качестве основы для проектирования схемы БД пригодно любое из минимальных покрытий, поскольку все они эквивалентны (совпадают их замыкания).

В алгоритме X_F^+ – замыкание множества атрибутов X на множестве зависимостей F вычисляется следующим образом:

Шаг 0. Начальное множество $X^0 = X$ (рефлексивность).

Шаг i ($i=1,2,\dots$). Выполняется итерация: просматриваются все функциональные зависимости из F . Если для какой-либо зависимости $Z \rightarrow Y \in F$, $Z \subseteq X^{i-1}$, тогда $X^i = X^{i-1} \cup Y$ и переход к следующей итерации. Поскольку, $X^{i-1} \subseteq X^i \subseteq U$, то алгоритм за конечное число шагов сойдется. Если на каком-то шаге после

просмотра всех функциональных зависимостей оказалось, что $X^{i-1}=X^i$ (не было сделано ни одной подстановки), тогда $X^+_F=X^i$. Конец построения.

Этап 3. Построить схему базы данных реляционного типа, удовлетворяющую требованиям третьей нормальной формы, свойству соединения без потерь информации и сохраняющую зависимости. Построить минимальное множество связей между отношениями БД

Полученное на этапе 2 минимальное покрытие множества функциональных зависимостей преобразуется следующим образом: объединяются зависимости с совпадающими левыми частями. Например, зависимости $X \rightarrow A_i$ и $X \rightarrow A_j$ объединяются в зависимость $X \rightarrow A_i A_j$.

Замечание. На практике зависимости $X \rightarrow A_i$ и $X \rightarrow A_j$ могут иметь различную область определения (одна зависимость имеет место для одного подмножества объектов, другая – для другого). Тогда эти зависимости объединять нельзя.

Далее из полученных зависимостей формируются отношения. Например, зависимость $A_i A_j \rightarrow A_l A_k A_m$ служит основанием для формирования отношения:

<u>A_i</u>	<u>A_j</u>	A_l	A_k	A_m
-------------------------	-------------------------	-------	-------	-------

где $A_i A_j$ – первичный ключ отношения.

Полученным отношениям присваиваются наименования. Если семантика атрибутов определена однозначно и функциональные зависимости построены без ошибок, то отношения будут иметь однозначную семантику и вводимое наименование отношения должно соответствовать этой семантике.

Такой способ формирования отношений гарантирует сохранение функциональных зависимостей и выполнение условий третьей нормальной формы (3НФ). При этом не гарантируется выполнение свойства соединения без потерь информации (зависимость соединения). Для проверки этого свойства формируем обобщенный ключ (суперключ), который функционально определяет все атрибуты из множества U : множество атрибутов $X \subseteq U$, является обобщенным ключом для U , если $X \rightarrow A_1, A_2, \dots, A_n \in F^+$; и для любого $Y \subset X$ (Y – истинное подмножество X) выполнено $Y \rightarrow A_1, A_2, \dots, A_n \notin F^+$.

Пусть X – обобщенный ключ. Если какое-либо отношение содержит атрибуты X (в качестве ключа), то совокупность сформированных отношений (схема БД) обладает свойством соединения без потерь информации (теорема 5.8 [1]). Если такого отношения нет, необходимо выполнить проверку по следующему алгоритму.

Исходные данные: декомпозиция $\rho(R_1, R_2, \dots, R_k)$ схемы отношения R , определенной на множестве атрибутов $U=(A_1, A_2, \dots, A_n)$ и множество функциональных зависимостей F .

Шаг 1. Строим начальную таблицу, содержащую k строк и n столбцов. На пересечении i -ой строки и j -ого столбца ставим символ a_j , если атрибут A_j принадлежит отношению R_i , иначе – b_{ij} .

Шаг 2. Выполняем очередную итерацию – последовательно просматриваем все зависимости из F . Для текущей зависимости $X \rightarrow Y \in F$ выбираем строки из таблицы, которые совпадают по атрибутам X (совпадать могут символы a_j и b_{ij}). Если для какой-либо строки атрибут A_{j^*} , принадлежащий Y , имеет значение a_{j^*} , то значение b_{ij^*} в выделенных строках заменяется на a_{j^*} . Если для всех строк атрибут A_{j^*} , принадлежащий Y , имеет различные значения b_{ij^*} , то значения b_{ij^*} в выделенных строках отождествляются (заменяются на одно какое-либо значение $b_{i;j^*}$ из выделенных строк). Если после очередной итерации появилась строка, состоящая из одних a_j , то декомпозиция ρ обладает свойством соединения без потерь информации. Если после очередной итерации не было сделано ни одной подстановки и в таблице отсутствует строка, состоящая из a_j , то декомпозиция не обладает свойством соединения без потерь информации. Конец алгоритма.

Если декомпозиция не обладает свойством соединения без потери информации, то ее необходимо дополнить новым отношением, состоящим полностью из атрибутов обобщенного ключа. Такая операция гарантирует, что декомпозиция будет удовлетворять указанному свойству, но при этом порождаются дополнительные проблемы.

Проблемы обобщенного ключа и способы их решения:

1. Совокупность атрибутов в обобщенном ключе X не обладает свойством однозначной семантической интерпретации: этому отношению нельзя присвоить однозначное имя. Решения:

а) выявляются потерянные функциональные зависимости на атрибутах X .

б) дополняются новые атрибуты, либо меняется семантика существующих атрибутов в X , для установления новых функциональных зависимостей на атрибутах X . После изменений, сделанных в пунктах а) и б) необходимо сделать соответствующие изменения на предыдущих этапах и снова вернуться к построению декомпозиции.

в) выявляется многозначная зависимость на атрибутах X и осуществляется декомпозиция отношения X (см. далее).

2. Если отношение X оказалось интерпретируемым, то оно может быть не технологичным: на предприятии отсутствует служба, которая отвечала бы за сопровождение данных в этом отношении. Решение:

а) сформировать новую схему документооборота на предприятии.

б) придется признать, что на самом деле существует не одна, а несколько БД, и они не могут быть интегрированы.

Многозначные зависимости. Дано: схема отношения R , определенная на совокупности атрибутов $U = \{A_1, A_2, \dots, A_n\}$, пусть $W \subseteq U$, $Y \subseteq U$ и $W \cap Y = \emptyset$, $Z = R \setminus (W \cup Y)$.

Определение. Множество W мультиопределяет множество Y в контексте Z : $W \twoheadrightarrow Y(Z)$ (многозначная зависимость), если для произвольной реализации r

схемы R существует два кортежа $t_1, t_2 \in r$ таких, что $t_1[W] = t_2[W]$, то существует кортеж t_3 , для которого выполнено:

$$t_3[W] = t_1[W], t_3[Y] = t_1[Y], t_3[Z] = t_2[Z],$$

в силу симметрии существует кортеж t_4 :

$$t_4[W] = t_1[W], t_4[Y] = t_2[Y], t_4[Z] = t_1[Z].$$

Замечание. Множества атрибутов Y и Z обычно содержатся в обобщенном ключе, а множество W может оказаться за пределами обобщенного ключа X .

Основной признак (необходимый, но не достаточный) наличия многозначной зависимости в X является следствием определения: дополнение одного кортежа в реализацию X приводит к необходимости дополнения еще нескольких кортежей.

После определения многозначной зависимости отношение, соответствующее обобщенному ключу X , декомпозируется на два отношения R_s и R_p , содержащие атрибуты WY и WZ соответственно. Основным признаком правильности определения множества W является отсутствие возможности появления лишних кортежей в объединенной таблице:

$$R_s \bowtie R_p,$$

где $\bowtie$ – операция естественного соединения, т.е. таких кортежей, которые не имеют смысла в данной прикладной области. Это является гарантией, что после декомпозиции обобщенного ключа схема БД по прежнему удовлетворяет свойству соединения без потерь информации, но уже в рамках четвертой нормальной формы (4НФ)

Чаще всего отношения R_s и R_p , являются частью существующих отношений $R_{m(i,j)}$, сформированных по функциональным зависимостям:

$$R_s = \pi_{WY}(R_{m(s,1)} \bowtie R_{m(s,2)} \bowtie \dots \bowtie R_{m(s,d)})$$

и

$$R_p = \pi_{WZ}(R_{m(p,1)} \bowtie R_{m(p,2)} \bowtie \dots \bowtie R_{m(p,g)})$$

(совокупности отношений в скобках также должны удовлетворять свойству соединения без потери информации).

Такие отношения, R_s и/или R_p , считаем существующими и удаляем их из схемы БД.

Завершающим построением этого этапа является установление связей между сформированными отношениями. Формальным основанием для установления связей являются зависимости включения:

Определение. Пусть $R_i[A_1, \dots, A_m]$ и $R_j[B_1, \dots, B_p]$ – схемы отношений (не обязательно различные), $V \subseteq \{A_1, \dots, A_m\}$ и $W \subseteq \{B_1, \dots, B_p\}$, $|V| = |W|$, тогда объект $R_i[V] \subseteq R_j[W]$ называется зависимостью включения, если $\pi_V(R_i) \subseteq \pi_W(R_j)$.

В определении $|V|$ – мощность множества V , $\pi_V(R_i)$ – проекция отношения R_i по атрибутам V .

Условие $V=W$ является необходимым для установления связи. Такой вид зависимостей включения называется типизированными. Это дополнительное ограничение вполне согласуется с общепринятым свойством связей на схеме

БД: связи отражают количественное соотношение кортежей в отношениях и не обладают какой-либо семантикой. Необходимость связывания различных по смыслу атрибутов, скорее всего, является признаком потери какой-либо функциональной зависимости для связываемых атрибутов, либо, зависимость включения устанавливается для атрибутов-синонимов. То есть, решение семантических проблем на предварительном этапе проектирования схемы базы данных в большинстве случаев позволяет избавиться от необходимости использования нетипизированных зависимостей включения.

Введем обозначения: $PK(R_i)$ или просто $PK(i)$ – множество атрибутов, являющихся первичным ключом в отношении R_i ; $L_1(i,j)$ – связь 1:1 от R_i к R_j , где R_i главное отношение; $L_M(i,j)$ – связь 1:M от R_i к R_j , где R_i главное отношение; $L(i,j)$ – связь 1:1 либо 1:M от R_i к R_j , где R_i главное отношение.

Определение. Между отношениями R_i и R_j существует связь $L_1(i,j)$, если $PK(R_i) \subseteq PK(R_j)$ и для любых реализаций R_i и R_j , выполнено $\pi_X(R_i) \subseteq \pi_X(R_j)$, где $X = R_i \cap R_j$.

Определение. Между отношениями R_i и R_j существует связь $L_M(i,j)$, если $PK(R_i) \not\subseteq PK(R_j)$ и $PK(R_i) \subseteq R_j$.

Заметим, что ограничение целостности, задаваемое связью $L_M(i,j)$ подразумевает $\pi_V(R_i) \subseteq \pi_V(R_j)$, где $V = R_i \cap R_j$.

Определение. Связь $L(i,j)$ является избыточной, если задаваемые ею ограничения на значения атрибутов содержатся в других связях.

Утверждение. Связь $L(i,j)$ является избыточной, если и только если существуют связи:

$$L(i, m(1)), L(m(1), m(2)), \dots, L(m(p-1), m(p)), L(m(p), j) \quad (1)$$

и

$$PK(i) \subseteq R_{m(s)}, s=2,3,\dots,p, \quad (2)$$

где m – массив номеров отношений.

Следствие. Если существует (1) и $p=1$, то связь $L(i,j)$ избыточна.

Замечание. Последовательность нескольких связей не может быть избыточной. Допустим, что существуют последовательности (1) и $L(i,v)$, $L(v,j)$, $v \neq m(s)$, $s=1,2,\dots,p$. Удаление связей $L(i,v)$ и $L(v,j)$, независимо от условия (2), означало бы снятие ограничений с отношения R_v , накладываемых связью $L(i,v)$.

Поиск избыточных связей с использованием приведенных соотношений (1) и (2) является экспоненциальной задачей. Поскольку связи являются реализацией типизированных зависимостей включения, то к поиску избыточных связей применим алгоритм построения замыкания отношений, являющийся полиномиальным по времени, где L – множество всех связей и R_i^+ – замыкание отношения R_i . Проверяем на избыточность связь $L(i,j)$:

Шаг 0. $R_i^+ = R_i$ (рефлексивность зависимостей включения).

Шаг i ($i=1,2,\dots$). Выполняется итерация: просматриваются все связи $L(v,w) \in L$. Если для текущей связи $R_v \in R_i^+$ и $w=j$, то связь $L(v,w)$ избыточна (конец алгоритма). Если $R_v \in R_i^+$, $w \neq j$ и $R_w \notin R_i^+$, то выполняем подстановку: $R_i^+ = R_i^+ \cup R_w$. Если после выполнения одной итерации не было сделано ни одной подстановки, то связь $L(i,j)$ не является избыточной (конец алгоритма), в противном случае переход к следующей итерации.

С учетом того, что на каждой итерации к замыканию присоединяется не менее одного отношения (их k штук), рассмотренный алгоритм будет иметь полиномиальную сложность.

Этап 4. Составить три запроса к БД в терминах реляционной алгебры, используя форму универсального реляционного запроса:

- запрос с обобщенным ключом,
- запрос без обобщенного ключа,
- запрос на вычитание.

Совокупность отношений, входящих в запросы, должна локально удовлетворять свойству соединения без потерь информации.

Провести формальную оптимизацию запросов, используя свойства операций реляционной алгебры.

Этап 5. Сформировать описание физического представления данных, определив перечень основных файлов БД. Определить типы записей в основных файлах, типы и размеры полей для записей каждого типа. Определить наиболее подходящий метод доступа для каждого файла.

Замечание. При выборе метода доступа необходимо учитывать частоту обращения к файлу, типы индексируемых полей, частоту модификации записей файла, устойчивость к программным и аппаратным сбоям и т.д.

Результаты выполнения данного и следующего этапов должны явиться основанием для выбора СУБД.

Этап 6. Провести анализ особенностей реализации информационной системы, обратив особое внимание на детали работы приложений со схемой БД.

Пример

Рассмотрим прикладную область – "Чемпионат по футболу"

Перечень исходных документов:

- "Турнирная таблица",
- "Заявки команд на матч",
- "Судейский протокол",
- "Продажа билетов на матч".

Для сокращения объема примера опустим другие сведения, например, "Назначение судейской бригады на матч", "Обслуживающий персонал стади-

она" и т.д. Заметим, что правильное построение схемы БД позволит дополнить эту информацию без разрушения уже построенной схемы.

Этап 1. Общий перечень элементов данных (атрибутов). Список расширен номерами объектов, для которых отсутствует однозначная идентификация в прикладной области (команды, игры и т.д.).

1. Номер матча.
2. Номер команды хозяев матча.
3. Номер команды гостей матча.
4. Наименование команды.
5. Номер игрока в команде.
6. Номер члена команды.
7. Роль члена команды {тренер, защитник, массажист ...}.
8. Номер стадиона.
9. Название стадиона.
10. Город расположения стадиона.
11. Номер игрового события в матче.
12. Время игрового события.
13. Номер игрока, участвующего в игровом событии.
14. Номер команды игрока, участвующего в игровом событии.
15. ФИО игрока.
16. ФИО члена команды.
17. Название игрового события {гол, игра рукой ...}.
18. Номер зрительской трибуны.
19. Номер ряда на трибуне.
20. Номер места в ряду на трибуне.

Анализируем элементы данных на предмет наличия синонимов и омонимов. Элементы 2 и 3 – синонимы. Заменяем их парой элементов:

2. Номер команды.
3. Статус команды в матче {гость, хозяин}.

Это наиболее распространенная ошибка при проектировании, когда роль или функцию объекта выделяют в отдельный атрибут. Например, "ФИО водителя", "ФИО диспетчера" и т.д., тогда как нужны два атрибута: "ФИО сотрудника" и "Должность сотрудника".

Элементы 5 и 6 частично дублируют друг друга. Уточним семантику элемента 5 – "Игровой номер футболиста" (номер на футболке). После этого преобразования элемент 13 становится синонимом элемента 5. Элемент 13 удаляется. Аналогично элемент 14 является синонимом номера команды. Элементы 15 и 16 являются синонимами, причем элемент 16 имеет большую область определения, он и останется в перечне. Результирующий перечень элементов данных следующий:

1. Номер матча.
2. Номер команды.
3. Статус команды в матче {гость, хозяин}.
4. Наименование команды.

5. Игровой номер футболиста.
6. Номер члена команды.
7. Роль члена команды {тренер, защитник, массажист ...}.
8. Номер стадиона.
9. Название стадиона.
10. Город расположения стадиона.
11. Номер игрового события в матче.
12. Время игрового события.
13. ФИО члена команды.
14. Название игрового события {гол, игра рукой ...}.
15. Номер зрительской трибуны.
16. Номер ряда на трибуне.
17. Номер места в ряду на трибуне.
- Дополнительные атрибуты:
18. Номер названия игрового события.
19. Дата проведения матча.
20. Время проведения матча.

Этап 2. Для выделенных элементов строим множество функциональных зависимостей, используя обратный метод:

- $1 \leftarrow \emptyset$, обозначает, что первый элемент функционально ничем не определяется – функциональная зависимость отсутствует;
- $2 \leftarrow 1, 11$;
- $3 \leftarrow 1, 2$, если зафиксирован номер матча и номер команды, то статус команды в матче будет иметь точно одно значение;
- $4 \leftarrow 2$;
- $5 \leftarrow 1, 11$;
- $6 \leftarrow 2, 5$, если зафиксирован игровой номер футболиста, то номер члена команды будет иметь точно одно значение, в обратную сторону зависимости нет: не все члены команды являются футболистами;
- $7 \leftarrow 2, 6$;
- $8 \leftarrow 1$;
- $9 \leftarrow 8$;
- $10 \leftarrow 8$, не рекомендуется различными наименованиями определять другие элементы: зависимости $10 \leftarrow 9$ нет;
- $11 \leftarrow \emptyset$;
- $12 \leftarrow 1, 11$;
- $13 \leftarrow 2, 6$;
- $14 \leftarrow 1, 11$, хотя эта зависимость выполнена, ввод названия события всякий раз при фиксации события является трудоемким и чреват множеством ошибок, пронумеруем эти названия, дав возможность реализовать выбор из меню уже введенных названий;
- $14 \leftarrow 18$;
- $15 \leftarrow \emptyset$;

$16 \leftarrow \emptyset;$
 $17 \leftarrow \emptyset;$
 $18 \leftarrow 1, 11;$
 $19 \leftarrow 1;$
 $20 \leftarrow 1.$

Построив минимальное покрытие множества функциональных зависимостей, получим, что зависимость $1, 11 \rightarrow 14$ является избыточной.

Этап 3. Строим каноническую модель данных реляционного типа, удовлетворяющую свойству соединения без потерь информации и сохраняющую зависимости.

Объединяем зависимости с одинаковой левой частью. Результирующее множество имеет следующий вид:

$1, 2 \rightarrow 3;$
 $2 \rightarrow 4;$
 $1, 11 \rightarrow 2, 5, 12, 18;$
 $2, 5 \rightarrow 6;$
 $2, 6 \rightarrow 7, 13;$
 $1 \rightarrow 8, 19, 20;$
 $8 \rightarrow 9, 10;$
 $18 \rightarrow 14.$

Из полученных зависимостей формируем отношения, присваивая им наименования и подчеркивая ключевые элементы данных (первичные ключи отношений).

R_1 =Статусы команд в матчах

<u>1. Номер матча</u>	<u>2. Номер команды</u>	3. Статус команды в матче
-----------------------	-------------------------	---------------------------

R_2 =Команды

<u>2. Номер команды</u>	4. Наименование команды
-------------------------	-------------------------

R_3 =Игровые события

<u>1. Номер матча</u>	<u>11. Номер игрового события в матче</u>	2. Номер команды	5. Игровой номер футболиста
12. Время игрового события		18. Номер названия игрового события	

R_4 =Распределение игровых номеров

<u>2. Номер команды</u>	<u>5. Игровой номер футболиста</u>	6. Номер члена команды
-------------------------	------------------------------------	------------------------

R_5 =Списочный состав команд

<u>2. Номер команды</u>	<u>6. Номер члена команды</u>	<u>7. Роль члена команды</u>	<u>13. ФИО члена команды</u>
-------------------------	-------------------------------	------------------------------	------------------------------

R_6 =Расписание матчей

<u>1. Номер матча</u>	<u>8. Номер стадиона</u>	<u>19. Дата проведения матча</u>	<u>20. Время проведения матча</u>
-----------------------	--------------------------	----------------------------------	-----------------------------------

R_7 =Стадионы

<u>8. Номер стадиона</u>	<u>9. Название стадиона</u>	<u>10. Город расположения стадиона</u>
--------------------------	-----------------------------	--

R_8 =Справочник игровых событий

<u>18. Номер названия игрового события</u>	<u>14. Название игрового события</u>
--	--------------------------------------

Обобщенный ключ следующий:

1. Номер матча.
11. Номер игрового события в матче.
15. Номер зрительской трибуны.
16. Номер ряда на трибуне.
17. Номер места в ряду на трибуне.

Отношение, сформированное для обобщенного ключа, имеет вид

<u>1. Номер матча</u>	<u>11. Номер игрового события в матче</u>	<u>15. Номер зрительской трибуны</u>	<u>16. Номер ряда на трибуне</u>	<u>17. Номер места в ряду на трибуне</u>
-----------------------	---	--------------------------------------	----------------------------------	--

Полученное отношение имеет приемлемую содержательную интерпретацию, однако совершенно не технологично. Действительно, при регистрации очередного игрового события оператор должен дублировать сведения о нем для всех проданных билетов на матч. Это, в свою очередь, является признаком наличия многозначной зависимости. Зависимость имеет вид

$1 \rightarrow \rightarrow 11(15,16,17)$ или $1 \rightarrow \rightarrow 15,16,17(11)$

В данном случае элемент данных в левой части зависимости находится в обобщенном ключе, что необязательно для других прикладных областей. Отношение, соответствующее обобщенному ключу, декомпозируем на два отношения:

<u>1. Номер матча</u>	<u>11. Номер игрового события в матче</u>
-----------------------	---

R_9 =Регистрация проданных билетов

<u>1. Номер матча</u>	<u>15. Номер зрительской трибуны</u>	<u>16. Номер ряда на трибуне</u>	<u>17. Номер места в ряду на трибуне</u>
-----------------------	--------------------------------------	----------------------------------	--

После этой операции заметим, что первое отношение является частью отношения R_3 , а новое отношение R_9 является не только интерпретируемым, но и технологичным (заполняется электронным кассовым аппаратом при продаже билетов на матч).

Установим связи между сформированными отношениями. Обозначим связь типа 1:1 символом $\rightarrow$, а связь 1:M символом $\Rightarrow$:

$$R_1 \Rightarrow R_3$$

$$R_2 \Rightarrow R_1$$

$$-R_2 \Rightarrow R_3$$

$$-R_2 \Rightarrow R_4$$

$$R_2 \Rightarrow R_5$$

$$R_4 \Rightarrow R_3$$

$$R_5 \Rightarrow R_4$$

$$R_6 \Rightarrow R_1$$

$$-R_6 \Rightarrow R_3$$

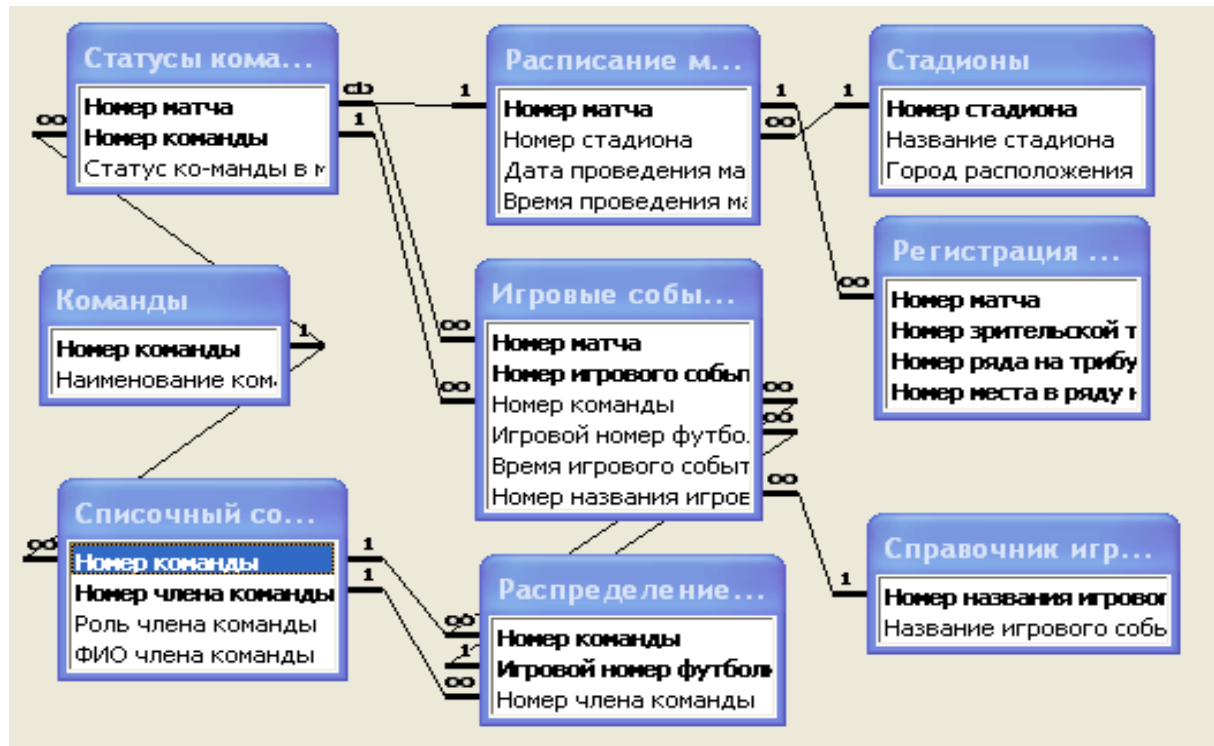
$$R_6 \Rightarrow R_9$$

$$R_7 \Rightarrow R_6$$

$$R_8 \Rightarrow R_3$$

Анализируя связи на схеме, получаем, что связи $R_2 \Rightarrow R_3$ и $R_6 \Rightarrow R_3$ являются избыточными и подлежат удалению. Проблематичной для удаления является связь $R_2 \Rightarrow R_4$ из-за того, что одна из заменяющих ее связей: $R_2 \Rightarrow R_5$ и $R_5 \Rightarrow R_4$, а именно $R_2 \Rightarrow R_5$, имеет область определения "Для всех членов команды". Тогда как удаляемая связь имеет область определения только "Для игроков команды". Однако вторая связь: $R_5 \Rightarrow R_4$, также имеет область определения только "Для игроков команды". Следовательно, удаление связи $R_2 \Rightarrow R_4$ не нарушит ссылочной целостности на данные.

Результирующая схема БД имеет следующее графическое представление:



Этап 4. В соответствии с заданием сформулируем и формализуем три запроса.

А) Запрос с обобщенным ключом. Обобщенный ключ содержится в отношениях R_3 и R_9 . Поскольку отношение R_9 не имеет содержательных атрибутов (а таковым, например, мог быть атрибут "Цена билета"), то запрос будет носить формальный характер: «Получить перечень всех номеров зрительских мест на трибуне с номером "6" в матчах в городе "Омск", зрители на которых были свидетелями игрового события "Вне_игры" в период 20.07.2007 по 20.09.2007».

Исходный запрос:

$$\pi_{\langle 16 \rangle, \langle 17 \rangle}(\sigma_{\langle 15 \rangle = 6, \langle 10 \rangle = \text{"Омск"}, \langle 14 \rangle = \text{"Вне_игры"}, \langle 19 \rangle \geq 20.07.2007, \langle 19 \rangle \leq 20.09.2007}(R_3 \bowtie R_6 \bowtie R_7 \bowtie R_8 \bowtie R_9)).$$

Оптимизированный запрос:

$$\pi_{\langle 16 \rangle, \langle 17 \rangle}(\pi_{\langle 18 \rangle}(\sigma_{\langle 14 \rangle = \text{"Вне_игры"}}(R_8)) \bowtie \pi_{\langle 8 \rangle}(\sigma_{\langle 10 \rangle = \text{"Омск"}}(R_7)) \bowtie \bowtie \pi_{\langle 1 \rangle, \langle 8 \rangle}(\sigma_{\langle 19 \rangle \geq 20.07.2007, \langle 19 \rangle \leq 20.09.2007}(R_6)) \bowtie \bowtie \pi_{\langle 1 \rangle, \langle 18 \rangle}(R_3) \bowtie \pi_{\langle 1 \rangle, \langle 16 \rangle, \langle 17 \rangle}(\sigma_{\langle 15 \rangle = 6}(R_9))).$$

Последовательность итерирования отношений R_8 , R_7 , R_6 , R_3 , R_9 гарантирует, что в системном буфере будет присутствовать минимальное количество данных.

Б) Запрос без обобщенного ключа. Получить список игроков команды «Иртыш», забивших голы.

Исходный запрос:

$$\pi_{\langle 13 \rangle}(\sigma_{\langle 4 \rangle = \text{"Иртыш"}, \langle 14 \rangle = \text{"Гол"}}(R_2 \bowtie R_3 \bowtie R_4 \bowtie R_5 \bowtie R_8)).$$

Поскольку отношения в запросе не содержит обобщенного ключа, то проверяем выполнение свойства соединения без потери информации для заданной совокупности отношений. Замыкание множества атрибутов, соответствующих ключу отношения R_3 , на множестве сохраненных зависимостей в отношениях запроса содержит все атрибуты из этих отношений, поэтому искомое свойство для выбранной совокупности отношений будет выполнено. В общем случае, когда ключ не содержится в одном отношении, проверку свойства необходимо осуществлять по алгоритму. Заметим, что аналогичное свойство должно быть выполнено для отношений первого запроса. Однако, наличие в нем обобщенного ключа (суперключа) существенно упрощает проверку этого свойства. Достаточно построить замыкание этого ключа на множестве сохраненных в декомпозиции зависимостей.

Оптимизированный запрос:

$$\pi_{<13>}(\pi_{<18>}(\sigma_{<14>="Гол"(R_8)}) \bowtie \pi_{<2>}(\sigma_{<4>="Иртыш"(R_2)}) \bowtie \pi_{<2><6><13>}(\pi_{<2><5><18>}(R_3)) \bowtie R_4 \bowtie R_5).$$

В) Запрос на вычитание. Этот класс запросов предназначен для получения информации о событиях, которые могли произойти, но не произошли (не зафиксированы в базе данных). Получить ФИО члена команды и название команды для нападающих, которые ни разу не забили гол.

Исходный запрос:

$$\pi_{<13><4>}(\sigma_{<7>="Нападающий"(R_2 \bowtie R_5)}) / \pi_{<13><4>}(\sigma_{<7>="Нападающий", <14>="Гол"(R_2 \bowtie R_3 \bowtie R_4 \bowtie R_5 \bowtie R_8)}).$$

Оптимизированный запрос:

$$\pi_{<13><4>}(\pi_{<2><7><13>}(\sigma_{<7>="Нападающий"(R_5)}) \bowtie \pi_{<2><4>}(R_2)) / \pi_{<13><4>}(\pi_{<2><7><13>}(\sigma_{<7>="Нападающий"(R_5)}) \bowtie \pi_{<2><4>}(R_2) \bowtie R_4 \bowtie \pi_{<2><5><18>}(R_3) \bowtie \pi_{<18>}(\sigma_{<14>="Гол"(R_8)})).$$

Этап 5. Выбор способа физического представления данных.

Для ускорения поиска данных в нескольких таблицах одновременно довольно часто используется прием денормализации отношений. То есть, на логическом уровне (на схеме) объединяются отношения, сопоставимые по размерам и часто используемые совместно в запросах. Большинство СУБД не имеют возможности сформировать физическое представление данных, отличное от логического описания (схемы базы данных). Другими словами, состав полей физических записей должен соответствовать составу элементов данных логических записей. В этом случае денормализация отношений имеет довольно слабое оправдание. Однако, данную проблему нужно решать за счет использования СУБД, позволяющих реализовать совместное хранение отношений. Примером такой СУБД является ORACLE, которая позволяет организовать совместное хранение отношений в виде кластера. Ограничением на кластер является один набор элементов данных (ключей кластера), по которому организуется совместное хранение. В нашем случае потребуются другая воз-

возможность СУБД: раздельное хранение отношений на различных серверах (см. этап 6).

Введем обозначения для типов данных: sN – символьная константа длины N ; inN – целая константа длины N байтов; d – агрегат данных "дата" (день.месяц.год); t – агрегат данных "время" (час:минута:секунда); $fn.m$ – финансовая константа, где n – количество разрядов для указания цены в рублях, m – добавочные разряды для копеек. Пусть $k(tp)$ – элемент данных с номером k и типом tp .

Физические записи, с учетом высказанных замечаний и введенных обозначений, могут иметь следующий вид:

1. Статус_команды (реализация R_1):
 $1(in2), 2(in1), 3(s6);$
2. Команды (реализация R_2):
 $2(in1), 4(s15);$
3. Игровые_события (реализация R_3):
 $1(in2), 11(in2), 2(in1), 5(in1), 12(t), 18(in1);$
4. Игровые_номера (реализация R_4):
 $2(in1), 5(in1), 6(in1);$
5. Составы_команд (реализация R_5):
 $2(in1), 6(in1), 7(s12), 13(s20);$
6. Расписание (реализация R_6):
 $1(in1), 8(in1), 19(d), 20(t);$
7. Стадионы (реализация R_7):
 $8(in1), 9(s15), 10(s15);$
8. Справочник_событий (реализация R_8):
 $18(in1), 14(s15);$
9. Продажа_билетов (реализация R_9):
 $1(in2), 15(in1), 16(in1), 17(in1);$

В соответствии с установленными типами данных, частотой обновления данных в файлах, частотой поиска данных в файлах по соответствующим полям и т.д., выберем способы индексации основных файлов БД.

- R_1 – инвертированный (по атрибутам 1 и 2),
- R_2 – индексно-последовательный (по атрибуту 2),
- R_3 – инвертированный (по атрибутам 1, 2, 5, 11 и 18),
- R_4 – инвертированный (по атрибутам 2, 5 и 6),
- R_5 – инвертированный (по атрибутам 2 и 6),
- R_6 – инвертированный (по атрибутам 1 и 8),
- R_7 – индексно-последовательный (по атрибуту 8),
- R_8 – индексно-последовательный (по атрибуту 18),
- R_9 – В-дерево (по атрибуту 1),

Наиболее затратной по памяти является индексация файлов R_3 и R_4 . Однако эти отношения чаще всего будут встречаться в запросах, требующих соединения по одноименным атрибутам. Причем, ограничения селекции будут задаваться на атрибуты других отношений, следовательно, эти отношения бу-

дуг итерироваться раньше, и при выполнении естественного соединения с R_3 и R_4 их индексы будут активно использоваться. Альтернативой этих индексов могут быть два индекса соединения 1) для отношений R_1 и R_3 по атрибутам 1 и 2, и 2) для отношений R_3 и R_4 по атрибутам 2 и 5. Возможно будет полезен индекс соединения отношений R_1 , R_2 и R_5 по атрибуту 2.

Этап 6. Анализ особенностей реализации информационной системы.

Данные по всем матчам чемпионата должны храниться на центральном сервере комитета по организации чемпионата, кроме данных из отношения R_9 . Вместо него в отношении R_6 дополняется атрибут «Количество проданных билетов». Отношения R_1 , R_2 , R_5 , R_6 , R_7 , и R_8 заполняются на сервере перед началом проведения чемпионата. Отношение R_4 пополняется перед матчем по заявке от команды, с возможными изменениями в отношении R_5 . Отношение R_3 пополняется в течение проведения матча в оперативном режиме.

Отношение R_9 пополняется кассовыми аппаратами перед проведением матча.

Перед проведением матча оператор из судейской бригады формирует на своей рабочей станции пустую базу данных, и закачивает в нее с сервера отношение R_8 и сведения из отношений R_1 , R_2 , R_5 , R_6 , и R_7 , относящиеся к текущему матчу. В течение матча оператором в отношении R_3 регистрируются игровые события. При дополнении записи, соответствующей событию «гол», должен быть запущен триггер, пересчитывающий результаты матча и выводящий эти результаты на табло стадиона. Комментаторы матча могут быть снабжены рабочими станциями с установленными на них сервисами для сбора статистики.

Из сказанного следует, что на период проведения матча должна быть установлена непрерывная связь (выделенная линия) между рабочими станциями на стадионе и центральным сервером комитета по организации чемпионата.

Перечень типовых заданий

1. Прикладная область: "Абитуриент". Документы: "Карточка абитуриента", "Расписание вступительных экзаменов", "Экзаменационная ведомость".

Вариант 1.1. Абитуриент может поступать на одну специальность (фиксированное расписание экзаменов для потоков).

Вариант 1.2. Абитуриент может поступать на несколько специальностей одновременно (индивидуальный график сдачи экзаменов для каждого абитуриента).

2. Прикладная область: "Факультет". Документы: "Расписание занятий", "Экзаменационная ведомость", "Приказы по деканату".

3. Прикладная область: "Продовольственный магазин". Документы: "Чек", "Электронная регистрация упаковки товаров", "Накладная на поставку товаров".

4. Прикладная область: "Мастерская по ремонту бытовой техники". Документы: "Заявка на ремонт оборудования", "Заявка от мастера на склад",

"Накладная на поставку запчастей". Замечание: при поступлении запчастей на склад им присваиваются инвентарные номера. Для крупных изделий индивидуальные номера, для мелких – номера на коробки. Со склада мелкие детали выдаются коробками.

5. Прикладная область: "Домоуправление". Документы: "Реестр жильцов", "Заявка на проведение ремонта в квартире", "Табель сотрудников домоуправления".

6. Прикладная область: "Общественный транспорт". Документы: "Расписание движения транспорта", "Электронная регистрация транспорта на остановках", "Табель сотрудников предприятия". Замечание: обязательно присутствие двух атрибутов – "Плановое время прибытия на остановку" и "Фактическое время прибытия на остановку"

7. Прикладная область: "Библиотека". Документы: "Каталог непериодических изданий", "Регистрация выданных книг", "Табель сотрудников библиотеки". Замечания: 1) в библиотеке могут быть несколько экземпляров одной и той же книги, различающиеся только инвентарными номерами; 2) у одной книги может быть несколько авторов и у одного автора несколько книг.

8. Прикладная область: "Общественное питание". Документы: "Чек", "Меню", "Заявка от бригады поваров на склад", "Накладная на поставку товаров". Замечание: по заявке на склад отдельно фиксируется выдача одинаковых продуктов, поступивших по различным накладным.

9. Прикладная область: "Обслуживание пассажиров на ж./д. вокзале". Документы: "Билет пассажира", "Бирка на багаж", "Расписание движения поездов". Рекомендуются ввести понятие подмаршрута и установить отношения между подмаршрутами.

10. Прикладная область: "Школа". Документы: "Расписание занятий", "Классный журнал", "Сведения об опекунах". Замечание: у одного опекуна может быть несколько учеников и у одного ученика может быть несколько опекунов. Рекомендуются ввести атрибут "Отношение родства", принимающий значения: отец, мать, бабушка и т.д.

11. Прикладная область: "Автостоянка". Документы: "Договор с клиентом", "Квитанция об оплате услуги автостоянки", "Электронная регистрация на въезде по госномеру автомобиля". Замечание: договор составляется на определенный период и, возможно, на несколько автомобилей, оплата услуг по договору может осуществляться частями.

12. Прикладная область: "Служба занятости". Документы: "Журнал регистрации безработных", "Заявка от предприятия", "Договор на переобучение группы безработных", "Резюме начальника отдела кадров принимающего предприятия".

13. Прикладная область: "Овощная база". Документы: "Накладная на поставку овощей на базу", "Накладная на отгрузку овощей в магазин", "Путевой лист автотранспорта". Замечание: при поступлении овощи загружаются в специальные ящики, причем, овощи от разных поставщиков в один ящик не загружаются.

14. Прикладная область: "Дом отдыха". Документы: "Выделение путевок домом отдыха на сезон", "Путевка", "Регистрация отдыхающих при заезде", "График мероприятий в ДО". Замечание: рекомендуется использовать атрибут "Отметка об участии в мероприятии".

15. Прикладная область: "Товарная станция". Документы: "Квитанция об приеме товара на отправку", "Опись товаров в контейнере", "Путевой лист на доставку товаров автотранспортом предприятия", "Накладная на доставку товара".

16. Прикладная область: "Поликлиника". Документы: "Карточка пациента", "Расписание работы врачей", "Талон на посещение врача". Замечание: В карточке больного фиксируется диагноз, принимаемые препараты и результат лечения.

17. Прикладная область: "Кинотеатр". Документы: "Регистрация предварительной продажи билетов", "Билет", "Расписание сеансов", "Накладная на поставку партии фильмов".

Библиографический список

1. Ульман Дж. Основы систем баз данных. – М.: Финансы и статистика, 1983. – 334 с.
2. Мейер Д. Теория реляционных баз данных. – М.: Мир, 1987. – 608 с.
3. Вейнеров О.М., Самохвалов Э.Н. Разработка САПР: В 10 кн. Кн. 4. Проектирование баз данных САПР. – М.: Высш. шк., 1990. – 144 с.
4. Дейт К. Введение в системы баз данных. – М.: Диалектика, 1998. – 782 с.
5. Тиори Т., Фрай Дж. Проектирование структур баз данных. – М.: Мир, 1985. Т 2. – 320 с.
6. Кузнецов С.Д. Основы баз данных. – М.: Интуит.ру, 2005. – 488 с.

